Start here: Scrapy

Scrapy is a Python framework for crawling websites and extracting structured data. You write a spider class that says which URLs to request and how to parse each response; Scrapy handles scheduling, concurrency, retries, deduplication, and exporting the items.

```
pip install scrapy        # Python 3.11+

# quotes.py
import scrapy


class QuotesSpider(scrapy.Spider):
    name = "quotes"
    start_urls = ["https://quotes.toscrape.com/"]

    def parse(self, response):
        for q in response.css("div.quote"):
            yield {
                "text": q.css("span.text::text").get(),
                "author": q.css("small.author::text").get(),
            }
        yield from response.follow_all(css="li.next a", callback=self.parse)

scrapy runspider quotes.py -O quotes.json
```

`-O` overwrites `quotes.json`; the log ends with `Stored json feed (N items) in: quotes.json`. This page covers version 2.19.0 (`scrapy/VERSION`). Examples were run against a local test server, not quotes.toscrape.com.

Day 1: use it in a project

```
scrapy startproject quotes      # creates quotes/scrapy.cfg + quotes/quotes/
cd quotes
scrapy genspider quotes_spider quotes.toscrape.com
scrapy list                     # spider names
scrapy crawl quotes_spider -O out.json
scrapy shell "https://quotes.toscrape.com/"   # try selectors interactively
```

`crawl`, `list`, `check`, `parse`, and `edit` only work inside a project directory; `runspider`, `shell`, `fetch`, and `genspider` work anywhere. `scrapy genrequest "curl ..."` turns a copied curl command into a `Request(...)` line.

Defaults that surprise

The generated `settings.py` is not the same as the library defaults (`scrapy/settings/default_settings.py`). A spider run with `runspider` behaves differently from the same spider inside a project.

Setting	Library default	New project's `settings.py`
`ROBOTSTXT_OBEY`	`False`	`True`
`CONCURRENT_REQUESTS_PER_DOMAIN`	`8`	`1`
`DOWNLOAD_DELAY`	`0`	`1` (seconds, ±50% jitter)
`FEED_MODE`	`None` (warns: will become `create`)	`"create"`
`FEED_EXPORT_ENCODING`	`None`	`"utf-8"`

Check the effective value with `scrapy settings --get NAME` from the directory you run in.

Gotchas

- `-o` in a project refuses to write an existing file. With `FEED_MODE = "create"` the log shows `ERROR: Not writing out.json because it already exists ... its items are lost`. Use `-O` to overwrite, or delete the file first.
- Off-domain requests vanish quietly. Links outside `allowed_domains` are dropped by `OffsiteMiddleware` with only a DEBUG line (`Filtered offsite request`). Check the `offsite/filtered` stat when a crawl stops early.
- Repeated URLs are dropped. The dupefilter skips any request it has already seen. Pass `dont_filter=True` on the `Request` to fetch it again. Requests from `start_urls` already set it.
- Every crawl opens two local servers: the Telnet console (`127.0.0.1:6023`, password in the log) and the `RemoteControl` HTTP server added in 2.19 (random localhost port, Bearer token, runs arbitrary Python inside the crawl). Turn them off with `TELNETCONSOLE_ENABLED = False` and `REMOTE_CONTROL_ENABLED = False` if that matters on shared machines.
- Without a reactor, HTTP goes through aiohttp. `TWISTED_REACTOR_ENABLED = False` is experimental, switches `http`/`https` to `AiohttpDownloadHandler`, and cannot be set per spider (`docs/topics/asyncio.rst`).
- `start()` replaced `start_requests()`. Since 2.13 the hook is `async def start(self)`. Keep a sync `start_requests()` only if the spider must also run on older Scrapy.

Run it from a script

```
from scrapy.crawler import AsyncCrawlerProcess

process = AsyncCrawlerProcess(settings={"FEEDS": {"items.json": {"format": "json"}}})
process.crawl(QuotesSpider)
process.start()  # blocks until the crawl finishes
```

Inside a project, pass `scrapy.utils.project.get_project_settings()` and the spider name instead.

Repository map

Path	What lives there
`scrapy/cmdline.py:184`	`execute()`, the `scrapy` CLI entry point
`scrapy/commands/`	one module per CLI command
`scrapy/crawler.py`	`Crawler`, `AsyncCrawlerProcess`, `AsyncCrawlerRunner`
`scrapy/core/`	engine, scheduler, downloader, scraper
`scrapy/downloadermiddlewares/`, `scrapy/spidermiddlewares/`	built-in middlewares
`scrapy/extensions/`	telnet, feed export, autothrottle, remote control
`scrapy/settings/default_settings.py`	every default value
`scrapy/templates/`	what `startproject` and `genspider` generate
`tests/`, `tests/mockserver/`	pytest suite and the local HTTP server tests crawl
`docs/`	Sphinx source for docs.scrapy.org; `sep/` holds design proposals

How a request flows

1. The engine (`ExecutionEngine`, `scrapy/core/engine.py:179`) pulls the spider's first requests from `Spider.start()`.
2. It hands them to the scheduler (`scrapy/core/scheduler.py:140`), which deduplicates and queues them.
3. The next request goes through downloader middlewares (`process_request`) to the `Downloader` (`scrapy/core/downloader/__init__.py:210`), which picks a download handler by URL scheme.
4. The response returns through downloader middlewares, then spider middlewares (`scrapy/core/spidermw.py`), to your callback.
5. The `Scraper` (`scrapy/core/scraper.py:142`) sends yielded items to item pipelines and yielded requests back to the scheduler.

Middlewares, pipelines, and extensions are enabled by dicts of `"import.path": order` in settings, merged with the `*_BASE` dicts in `default_settings.py`. The number sets the position in the chain; `None` disables a built-in.

Work on Scrapy itself

```
git clone https://github.com/scrapy/scrapy && cd scrapy
python -m venv .venv && . .venv/bin/activate
pip install -e . tox pre-commit
tox -e py -- tests/test_dupefilters.py     # args after -- go to pytest
pre-commit run --all-files
tox -e mypy && tox -e pylint
tox -e docs && tox -e docs-tests           # if you touched docs/docstrings
```

Plain `pytest tests/test_dupefilters.py` also works once the test deps from `tox.ini` (`[test-requirements]`) are installed; `pyproject.toml` adds `--reactor=asyncio` automatically.

Rule (from `AGENTS.md`)	Why
Prefix new non-public names with _	anything public becomes API you must keep
Build components with `scrapy.utils.misc.build_from_crawler()` in tests	it wires settings and signals the way a crawl does
Test spiders against `tests/mockserver`, using `example.com` URLs	tests never hit real sites
Don't edit `docs/news.rst`	release notes are written at release time
Never rebase or force-push a reviewed PR	PRs are squash-merged; history rewrites hide changes
Run the matching `min-*` tox env when relying on a dependency feature	CI pins the oldest supported versions

Read `docs/contributing.rst` before opening a pull request. Security reports go through `SECURITY.md`.